USE CASE STUDY REPORT
Restaurant Management

Group No.: Group 7
Student Names: Giastina Morano & Yi-Fan Liu

Executive Summary:

Restaurants are an extremely chaotic environment where critical operational data is often overlooked or lost entirely. To this day, there are still many restaurants that do not have an integrated data management system to help them, and thus create many internal problems. These problems include: inaccurate wait time estimates, unbalanced server workloads, lost reservations, and poor table utilization. This project will solve all those problems by creating a data management system, ultimately reducing customer dissatisfaction, revenue loss, and staff frustration that restaurants face without proper data support. This system will help with waitlist tracking, reservation management, table assignments, server workload balancing, guest preferences, and performance analytics for the servers, all extremely important aspects to create a seamless working environment in such a chaotic industry. With this operating system implemented, it will be simple to pull analytics from it, such as what the peak times are for guests coming into the restaurant, which servers take the most tables, which servers get tipped the most, and other key performance indicators, all information that was left behind without this system.

This system was modeled from real-world restaurant settings, allowing us to accurately conceptualize the model through EER and UML diagrams, followed by the relational model, where we defined our primary and foreign keys. After this, we could implement our database in MySQL efficiently and create queries to retrieve meaningful insights from the database. Overall, this system was implemented using MySQL for relational data management, MongoDB for NoSQL implementation, and Python for database connectivity and analytics.

I. Introduction

Business Problem Statement:

Restaurants face many operational challenges in managing things like customer flow, staffing allocation, and service quality. Most systems lead to inaccurate wait time estimates, unbalanced server workloads, lost reservations, and poor table utilization, which results in customer dissatisfaction, revenue loss, and staff frustration.

The Problem:

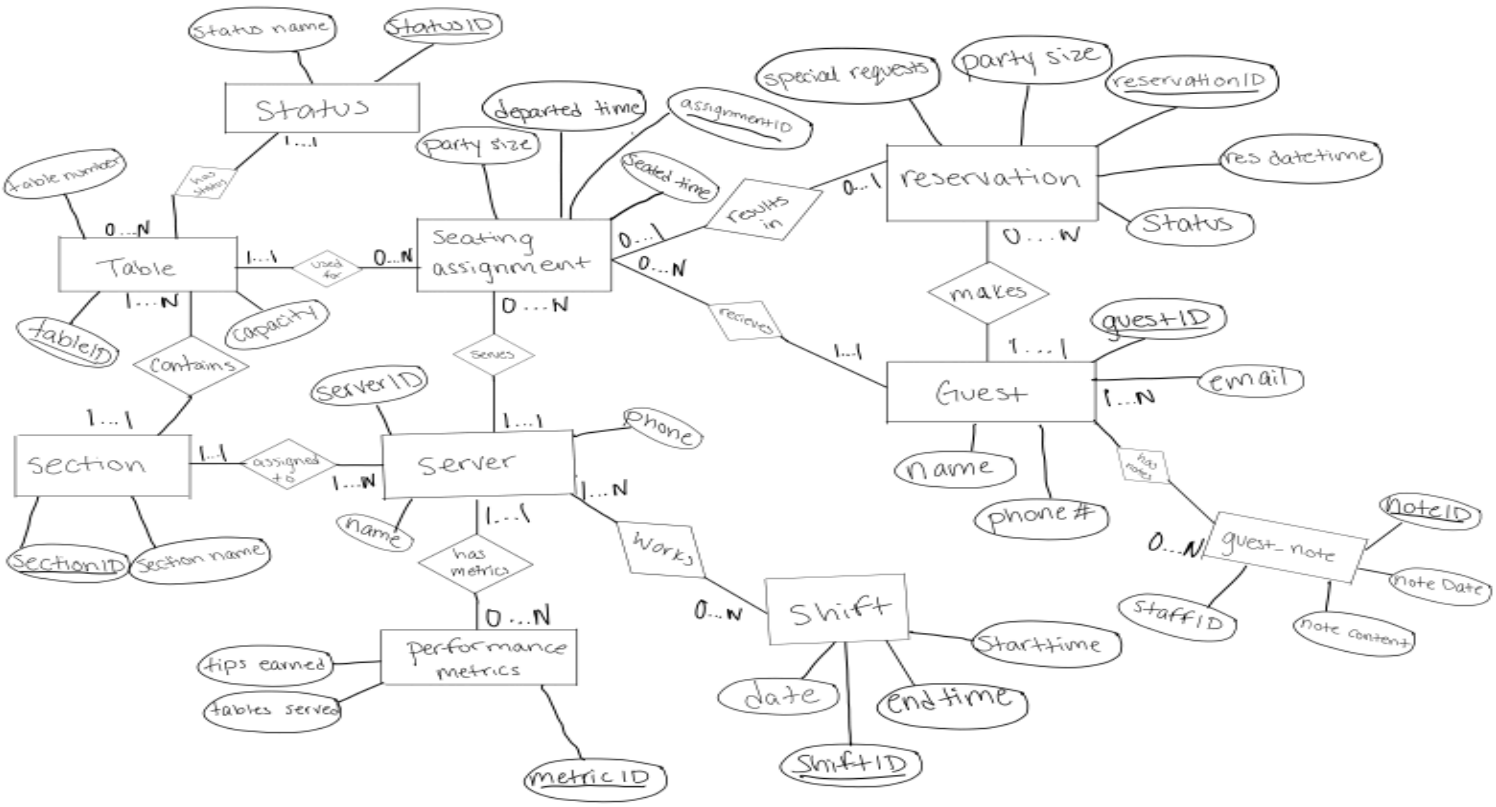
Restaurants need an integrated data management system that coordinates waitlist tracking, reservation management, table assignments, and server workload balancing while capturing guest preferences and enabling performance analytics.

DBMS Requirements:

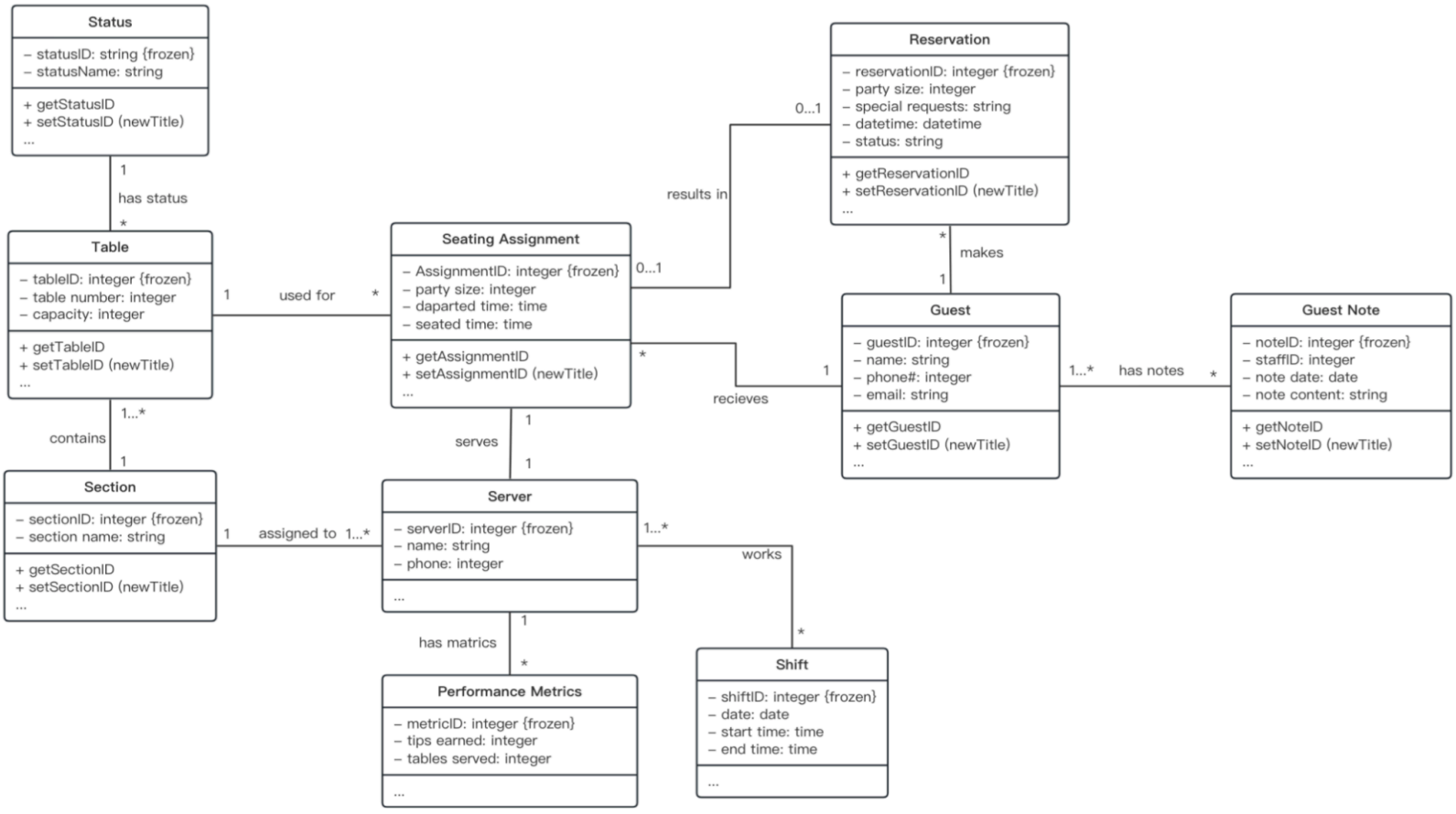
1. GUEST and RESERVATION: A guest can make zero to infinite reservations(for different dates/times); a reservation must be made by exactly one guest.
2. GUEST and GUEST_NOTE: A guest can have zero to infinite notes recorded by staff; a guest note must belong to exactly one guest
3. TABLE and SECTION: A table must belong to exactly one section; a section can contain multiple but must have at least one in its section.
4. SERVER and SECTION: A server must be assigned to exactly one section per shift; a section can be assigned to multiple but at least one server across different shifts.
5. SEVER and SHIFT: A server can work zero to infinite shifts; a shift must be worked by at least one server.
6. SEATING_ASSIGNMENT and TABLE: A seating assignment must be exactly one table; a table can have zero to infinite seating assignments over time(different parties throughout service).
7. SEATING_ASSIGNMENT and SERVICE: A seating assignment must be served by exactly one server; a server can have zero to many seating assignments during their shift.
8. SEATING_ASSIGNMENT and RESERVATION: A reservation can result in zero or one seating assignment; a seating assignment can be linked to zero or one reservation.
9. GUEST and SEATING_ASSIGNMENT: A guest can have zero to infinite seating assignments; a seating assignment must be before exactly one guest(or party).
10. SERVER and PERFORMANCE_METRICS: A server can have zero to infinite performance metric records; a performance metric record must belong to exactly one server.
11. TABLE and STATUS: A table must have exactly one current status at any time; a status type (available, occupied, reserved, bussed) can apply to zero to infinite table simultaneously.

II. Conceptual Data Modeling

EER Diagram



UML Diagram



III. Mapping Conceptual Model to Relational Model

Primary Key - Underlined **Foreign Key - *Blue Italics***

STATUS(statusID, status_name)
SECTION(sectionID, section_name)
TABLE(tableID, table_name, capacity, *statusID, sectionID*)
    statusID is a FK referring to statusID from STATUS, NOT NULL allowed
    sectionID is a FK referring to sectionID from SECTION, NOT NULL allowed
SEATING_ASSIGNMENT(assignmentID, party_size, seated_time, departed_time, *serverID*, *tableID*, *guestID*, *reservationID*)
    serverID is a FK referring to serverID from SERVER, NOT NULL allowed
    tableID is a FK referring to tableID from TABLE, NOT NULL allowed
    guestID is a FK referring to guestID from GUEST, NOT NULL allowed
    reservationID is a FK referring to reservationID from RESERVATION, NULL allowed
RESERVATION(reservationID, special_requests, party_size, date_time, status, *guestID*)
    guestID is a FK referring to guestID from GUEST, NOT NULL allowed
SERVER(serverID, name, phone, *sectionID*)
    sectionID is a FK referring to sectionID from SECTION, NOT NULL allowed
PERFORMANCE_METRICS(metricID, tables_serves, tips_earned, *serverID*)
    serverID is a FK referring to serverID from SERVER, NOT NULL allowed
SHIFT(shiftID, date, starttime, endtime)
WORKS(*serverID*, *shiftID*)
    serverID is a FK referring to serverID from SERVER, NOT NULL allowed
    shiftID is a FK referring to shiftID from SHIFT, NOT NULL allowed
GUEST(guestID, name, phone#, email)
GUEST_NOTE(noteID, note_date, note_content, staffID)
HAS_NOTES(*guestID*, *noteID*)
    guestID is a FK referring to guestID from GUEST, NOT NULL allowed
    noteID is a FK referring to noteID from GUEST_NOTE, NOT NULL allowed

IV. Implementation of the Relation Model via MySQL and NoSQL

MySQL Implementation:

The database was created in MySQL, and the following queries were performed:

Query 1: Find the reservation where the party size is greater than 3 and the datetime is '2026-03-21 17:00:00' (Simple query)

```
select * from reservation a
where a.status = 1
   and a.party_size > 3
   and a.date_time = '2026-03-21 17:00:00';
```

reservationID	special_requests	party_size	date_time	status	guestID
1		5	2026-03-21 17:00:00	1	7

Query 2: Retrieve the total tips of the server and their phone number, order by the total tips descending (Aggregate query)

```
select a.name, a.phone,
        sum(b.tips_earned) total_tips
 from server a
  join PERFORMANCE_METRICS b on a.serverid = b.serverid
group by a.name, a.phone
 order by sum(b.tips_earned) desc;
```

name	phone	total_tips
Bob	8570000002	45.00
Eve	8570000005	38.00
Alice	8570000001	36.00
Charlie	8570000003	27.00
David	8570000004	27.00

Query 3: Find all guests who have notes, show their note content and date, and their reservationID if they have one, and NULL if they don't (Inner Join and left join)

```
select g.guestName,
        g.guestID,
        gn.note_content,
        gn.note_date,
        r.reservationID
  from guest g
  join has_notes hn on g.guestID =
hn.guestID
  join guest_note gn on hn.noteID = gn.noteID
  left join reservation r on g.guestID = r.guestID;
```

guestName	guestID	note_content	note_date	reservationID
max	14	Regular customer	2026-03-20	36
will	20	Allergic to peanuts	2026-03-20	NULL
jen	19	Allergic to peanuts	2026-03-20	NULL
ellen	16	Regular customer	2026-03-20	NULL
zoe	5	Birthday celebration	2026-03-20	3

Query 4: Find reservations where the party size is larger than the average party size (Nested query)

```
select r.guestID,g.guestName,r.party_size
 from reservation r
  join guest g on r.guestID = g.guestID
where party_size >  (select AVG(party_size) from reservation);
```

guestID	guestName	party_size
7	jack	5
12	sue	6
18	jannie	6
18	jannie	6
4	angeline	5
9	sara	4
9	sara	5
9	sara	4
8	evan	4
9	sara	5
9	sara	4
8	evan	4

Query 5: Find guests who did not make a reservation (Correlated query with NOT EXISTS)

```
select guestID, guestName
  from guest
where NOT EXISTS( select guestID
                    from reservation
                   where reservation.guestID = guest.guestID
                  );
```

guestID	guestName
11	anne
15	katie
16	ellen
17	randy
19	jen
20	will
21	henry

Query 6: Find the biggest tip they've earned and the server name (Inner join query with ALL)

```
select b.name, a.tips_earned
  from PERFORMANCE_METRICS a
   join server b on a.serverid = b.serverid
where a.tips_earned >=
          ALL(select tips_earned from PERFORMANCE_METRICS);
```

name	tips_earned
Alice	20.00
Eve	20.00

Query 7: Find the reservations and guests on '2026-03-21' and '2026-03-22', where the party size is greater than 4 (Set operation - Union)

```
select a.date_time, a.party_size, b.guestID, b.guestName, b.phone, b.email
 from RESERVATION a
   join GUEST b on  a.guestID = b.guestID
where a.date_time like '2026-03-21%' and a.party_size > 4
union
select a.date_time, a.party_size, b.guestID, b.guestName, b.phone, b.email
 from RESERVATION a
   join GUEST b on  a.guestID = b.guestID
where a.date_time like '2026-03-22%' and a.party_size > 4;
```

date_time	party_size	guestID	guestName	phone	email
2026-03-22 19:00:00	6	12	sue	6170000012	sue@email.com
2026-03-22 15:00:00	6	18	jannie	6170000018	jannie@email.com
2026-03-23 17:00:00	6	18	jannie	6170000018	jannie@email.com
2026-03-23 15:00:00	5	4	angeline	6170000004	angeline@email.com

Query 8: Find guests who have guest notes and a reservation (Set operation - INTERSECT)

```
select g.guestName, gn.note_content
 from guest g
  join has_notes hn on g.guestID = hn.guestID
  join guest_note gn on hn.noteID = gn.noteID
INTERSECT
select g.guestName, gn.note_content
 from guest g
  join reservation r on g.guestID = r.guestID
  join has_notes hn on g.guestID = hn.guestID
  join guest_note gn on hn.noteID = gn.noteID;
```

guestName	note_content
max	Regular customer
zoe	Birthday celebration

Query 9: For each hour of the day, show the total number of guests dining (Subqueries in Select and From)

```
select s.name,
        (select COUNT(sa.reservationID)
           from seating_assignment sa
          where sa.serverID = s.serverID) as reservations_handeled
 from server s;
```

name	reservations_handeled
Alice	3
Bob	2
Charlie	3
David	2
Eve	0
Sam	0

NoSQL Implementation:

Query 1: Finds all reservations on March 21, 2026 (Simple query)

db.reservations.find({dateTime: {$regex: "^2026-03-21"}});

```
db.reservations.find({ dateTime: { $regex: "^2026-03-21" } });
```

Run

Result

```
[
  {
    _id: ObjectId('69d6b0b5492e85160f89b050'),
    reservationID: 1,
    notes: '',
    partySize: 5,
    dateTime: '2026-03-21 17:00',
    status: 'available',
    guestID: 7
  },
  {
    _id: ObjectId('69d6b0b5492e85160f89b051'),
    reservationID: 2,
    notes: '',
    partySize: 1,
    dateTime: '2026-03-21 19:00',
    status: 'available',
    guestID: 18
  },
  {
    _id: ObjectId('69d6b0b5492e85160f89b052'),
    reservationID: 3,
    notes: 'with wheel chair',
    partySize: 1,
    dateTime: '2026-03-21 18:00',
    status: 'available',
    guestID: 5
  },
  {
    _id: ObjectId('69d6b0b5492e85160f89b053'),
    reservationID: 4,
    notes: '',
    partySize: 3,
    dateTime: '2026-03-21 19:00',
    status: 'available',
    guestID: 7
  },
  {
    _id: ObjectId('69d6b0b5492e85160f89b054'),
    reservationID: 5,
    notes: '',
    partySize: 3,
    dateTime: '2026-03-21 15:00',
    status: 'available',
    guestID: 4
  }
]
```

Query 2: Finds all guests who have a note (Simple query)

db.guests.find({notes: {$not: {$size: 0}}});

```
db.guests.find({ notes: { $not: { $size: 0 } } });
```

Run

Result

```
[
  {
    _id: ObjectId('69d6b3b145a53c4ea189b041'),
    guestID: 5,
    name: 'zoe',
    phone: '6170000005',
    email: 'zoe@email.com',
    notes: [
      {
        noteID: 5,
        date: '2026-03-20',
        content: 'Birthday celebration'
      }
    ]
  }
]
```

Query 3: Find the reservations on 2026-03-21 with partySize greater than or equal to 3 and having notes

```
db.reservations.find({
  dateTime: { $regex: "^2026-03-21" },
  partySize: { $gte: 3 },
  notes: {$not: {$size: 0}}
})
```

```
12 db.reservations.find({
13   dateTime: { $regex: "^2026-03-21" },
14   partySize: { $gte: 3 },
15   notes: {$not: {$size: 0}}
16 })
17
```

Run

Result

```
[
  {
    _id: ObjectId('69d83469dff738654a89b050'),
    reservationID: 1,
    notes: '',
    partySize: 5,
    dateTime: '2026-03-21 17:00',
    status: 'available',
    guestID: 7
  },
  {
    _id: ObjectId('69d83469dff738654a89b053'),
    reservationID: 4,
    notes: '',
    partySize: 3,
    dateTime: '2026-03-21 19:00',
    status: 'available',
    guestID: 7
  },
  {
    _id: ObjectId('69d83469dff738654a89b054'),
    reservationID: 5,
    notes: '',
    partySize: 3,
    dateTime: '2026-03-21 15:00',
    status: 'available',
    guestID: 4
  }
]
```

Query 4: Find the count of reservations per guest and sorting descending order

```
db.reservations.aggregate([
  {
    $group: {
      _id: "$guestID",
      totalReservations: { $sum: 1 }
    }
  },
  {
    $sort: { totalReservations: -1 }
  }
])
```

```
6  db.reservations.aggregate([
7    {
8      $group: {
9        _id: "$guestID",
10       totalReservations: { $sum: 1 }
11     }
12   },
13   {
14     $sort: { totalReservations: -1 }
15   }
16 ])
17
```

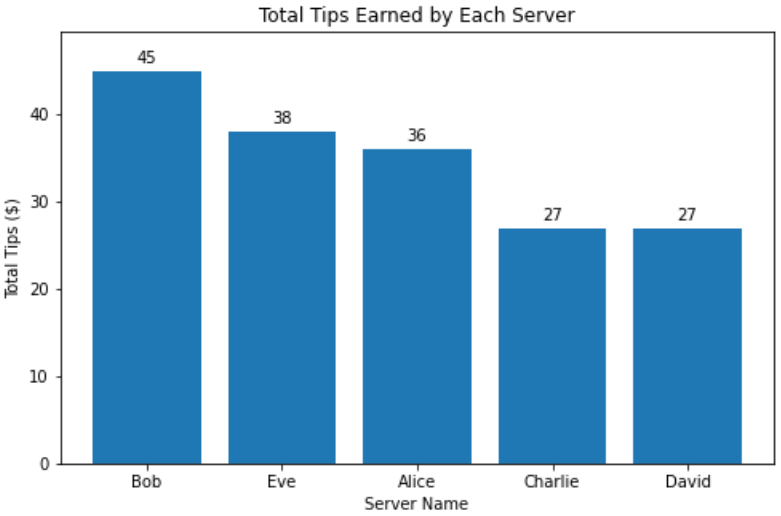
Run

Result

```
[
  { _id: 18, totalReservations: 3 },
  { _id: 7, totalReservations: 2 },
  { _id: 4, totalReservations: 2 },
  { _id: 9, totalReservations: 1 },
  { _id: 12, totalReservations: 1 },
  { _id: 5, totalReservations: 1 }
]
```

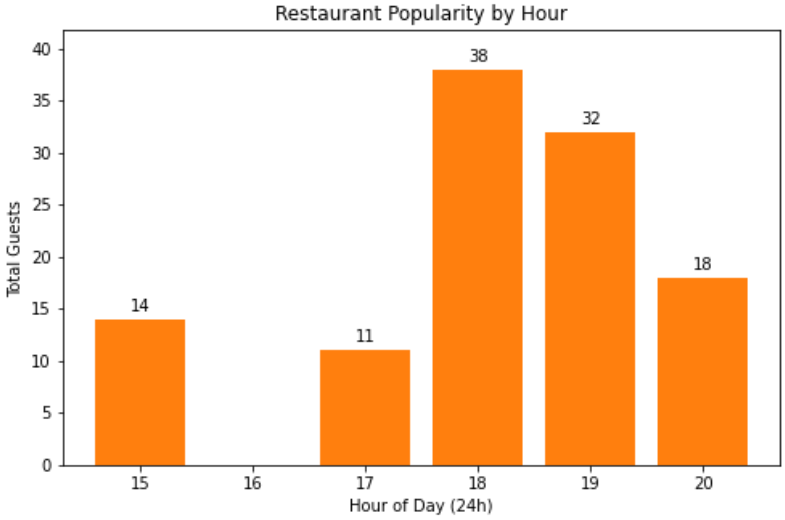
V. Database Access via Python

The database is accessed using Python, and the visualization of the analyzed data is shown below. The connection of MySQL to Python is done using “from sqlalchemy import create_engine”, followed by converting the list into a dataframe using the pandas library, and using matplotlib to plot the graphs for the analytics.



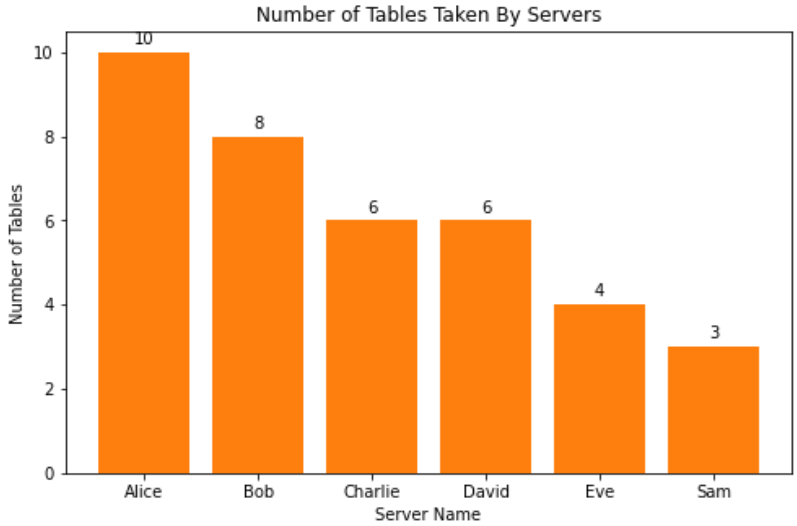
Query1: Total Tips Earned by Each Server

Bob earned the highest amount of tips at $45. Although Charlie and David earned the least tips, they still received $27 each. Thus, there is a difference of $18 between the highest earner and the lowest earner.



Query 2: Popularity of The Reservation in Each

Reservation popularity peaks between 18:00 and 19:00, and there are no recorded reservations at 16:00. However, the period around 16:00 remains a potential opportunity for growth or happy hour promotions due to the lack of bookings.



Query 3: Number of Tables taken by Servers

Overall, Alice has taken the most tables (10). Charlie and David have taken the same number of tables (6). Sam has taken the least tables (3).

Query4: Guest Notes

	guestName	guestID	note_content	note_date	reservationID
0	max	14	Regular customer	2026-03-20	36.0
1	will	20	Allergic to peanuts	2026-03-20	NaN
2	jen	19	Allergic to peanuts	2026-03-20	NaN
3	ellen	16	Regular customer	2026-03-20	NaN
4	zoe	5	Birthday celebration	2026-03-20	3.0

Type Markdown and LaTeX: α^2

Max and Zoe are 2 guests who have notes and have a reservation on 3/20/26. Max is a “regular customer,” and Zoe has a “birthday celebration” tonight. There are 3 remaining guests who have notes taken on them, with no reservation. Will and Jen are both “Allergic to peanuts,” and Ellen is a “regular customer”.

VI. Summary and Recommendation

The restaurant management database designed on MySQL is an industry-ready relational system that can be directly implemented in real-world settings. By using this DBMS, restaurants can achieve significant cost savings and improve customer satisfaction. This system also provides powerful analytics capabilities, a few of which we demonstrated in this report using Python.

Our implementation across MySQL, NoSQL, and Python highlights various critical aspects of restaurant operations, including reservation trends, server performance based on tips, and customer preferences/restrictions. As long as a restaurant continues to use this system and input data, the operations will become more efficient, providing owners with more accurate data to make informed business decisions.

Based on our MySQL and NoSQL queries, as well as the Python visualizations, we have a specific recommendation for staffing: we suggest scheduling more servers to work between 18:00 and 20:00 daily, as this is the peak dining period for customers. Additionally, if the restaurant wants to attract more customers before 18:00, they could offer early-bird specials or promotions, such as a "Happy Hour" with discounted drinks to help balance the guest flow throughout the day. Overall, this restaurant management system represents a complete and well-structured solution that is ready for implementation to help restaurants streamline their operations, improve guest experiences, and bring efficiency to one of the fastest-paced industries in the world.